

GarudaChip

Recursive-Language-Model agent + Autoresearch

self-improving prompt → RTL → GDSII, on a local model

RLM deep agents

pgvector + MinIO memory

autoresearch loop

System Design Document

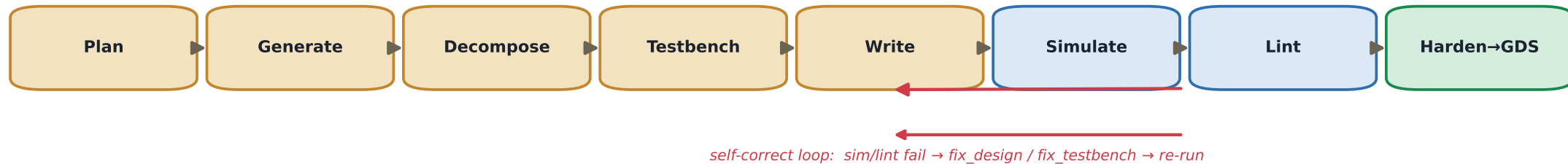
current architecture · knowledge-store coordination · the autoresearch upgrade

Local model only (Ollama qwen3.5) · gf180 PDK · LibreLane RTL→GDSII

1 · Current RLM design — fixed graph, every node a deep agent

as built today

The orchestration is a FIXED pipeline graph; each LLM node is upgraded into a Recursive-Language-Model (RLM) deep agent (planning + on-disk tools + sub-LLM delegation).



Anatomy of ONE deep-agent node

- write_todos — plan the multi-step work
- file tools — list / read_file_disk (SLICE) / grep_files /
- write_file_disk (auto-repair + compile-check on write)
- llm_query — delegate ONE focused sub-task to a fresh LLM call
- task — spawn a heavier sub-agent
- run_python / pip_install — compute LUTs / NN weights (data-driven)

RLM operating loop

- PEEK, don't swallow — grep / read slices of big context
- DELEGATE — sub-LLM calls & sub-agents keep the window small
- BUILD UP in files — RTL lives on disk, not in the prompt
- VERIFY — compile-on-write + static audit + closure compile
- ANCHOR-AND-ADAPT — copy the closest GitHub module, edit to spec

Deterministic gates (verilog_check) — make a 9B local model reliable

compile-on-write
(iverilog -g2012)

static cross-module
audit

closure compile
(top cone only)

multi-file
auto-fix

error→fix lesson
capture

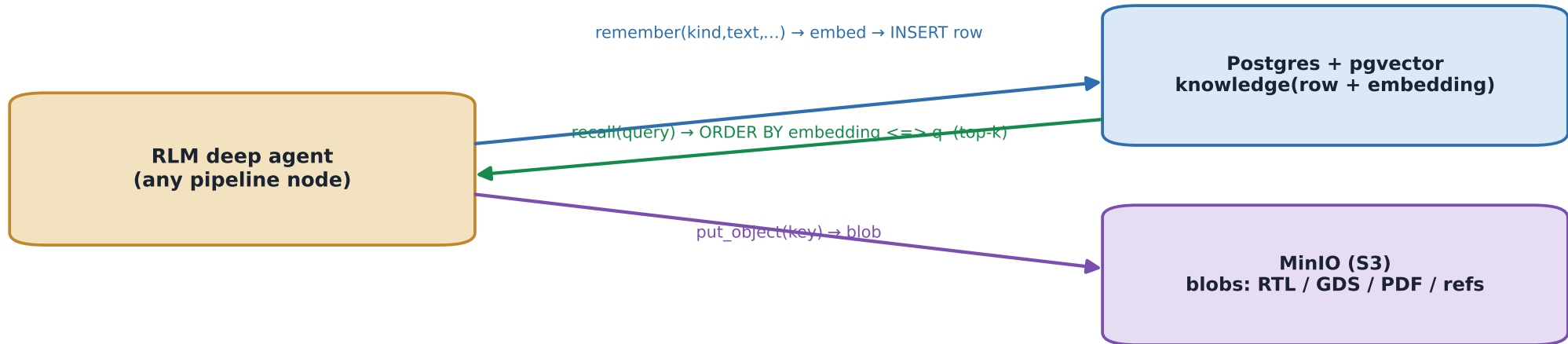
Key property: the model never holds the whole design at once — it peeks, delegates, and writes to disk, while deterministic checks catch errors at write-time, not 10 modules later.

Local model only (Ollama qwen3.5) · deepagents framework · the graph orchestrates, each node's brain is an RLM.

2 · Knowledge-store coordination — Postgres(pgvector) + MinIO

the RLM's long-term memory

Everything the agent sees or makes is stored two ways: the RAW FILE as a blob in MinIO, and a queryable ROW (with its embedding vector) in Postgres. One row carries catalog + vector, so semantic recall combines with SQL filters in a single query.



What gets stored (kind-tagged rows + blobs):

- design — every VERIFIED RTL design (reusable, recallable)
- code — RTL / TB source files
- fix — error→fix LESSONS (the self-improvement signal)
- research / paper / reference — crawled GitHub + papers digests
- gds — hardened layouts (ingested for tape-out reuse)
- ip — the IP library (peripherals, cores, memory, crypto ...)

How the recursive loop USES the store

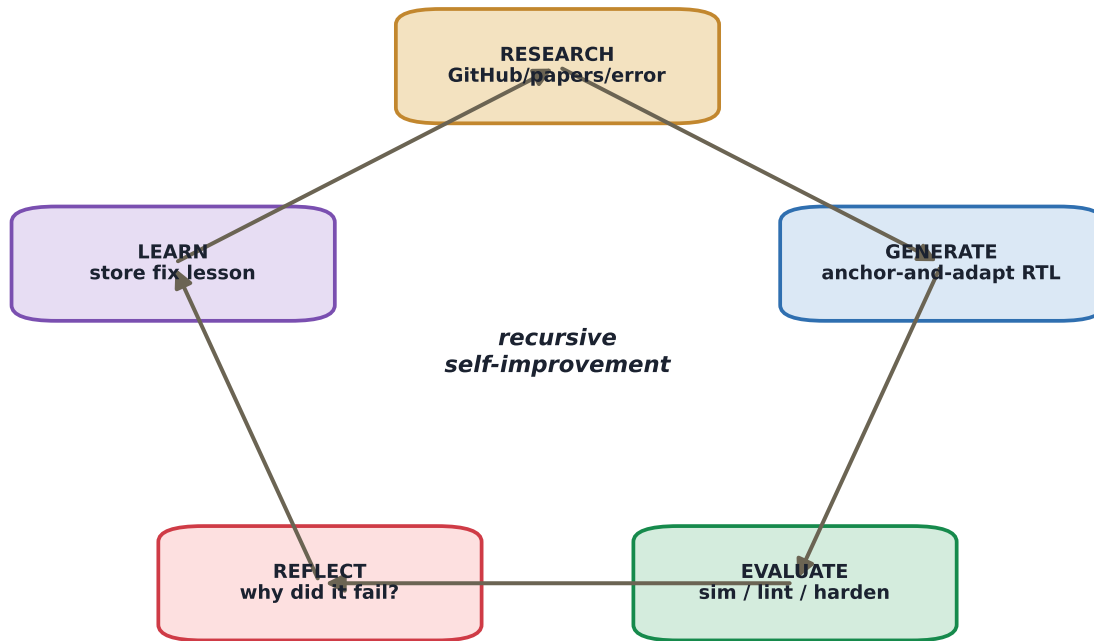
- BEFORE generate — recall the 3 closest error→fix
- lessons + the nearest verified design (the anchor)
- DURING generate — a broken→clean write saves a NEW
- fix lesson immediately (kind='fix')
- AFTER a verified run — ingest the whole design so the
- next prompt can recall it
- DEFENSIVE — if the DB/bucket is down, every call no-ops
- and the agent keeps running on the local file flow

Net effect: the store is the agent's MEMORY — recall makes each new design start from the closest prior knowledge; remember makes every solved problem permanent.

3 · The Autoresearch upgrade — self-improving RTL generation

auto-kernel-gen, applied to RTL

Insight: projects that auto-generate OPTIMAL KERNELS (AutoKernel, KernelBench, and Reflexion / Self-Refine self-improving agents) close a research→generate→evaluate→reflect→learn loop. RTL generation is the same shape — so we make the loop EXPLICIT and feed it from the store.



What changes vs. today

- PROACTIVE research — before generating, search GitHub + papers for the architecture AND its known pitfalls, crawl them (SearXNG + crawl4ai) into a research digest
- TARGETED research-on-failure — when sim/lint/harden fails, research the EXACT error message, not just the topic
- REFLECTION — turn each failure into a written lesson: symptom → broken pattern → correct fix
- ACCUMULATION — lessons + verified designs grow in the store, so the agent gets measurably better every run
- PROMOTION — a memory-keeper distills raw lessons into per-topic knowledge summaries (Tier-2 knowledge)

Why this works for RTL (same as for kernels)

Kernel codegen

search ops + perf tricks → emit kernel → benchmark → reflect on slow paths → cache

RTL generation (ours)

research design + pitfalls → adapt RTL → sim/lint/harden → reflect on errors → store fix

Verifiable reward signal (compile / sim-pass / DRC-clean / timing-met) is what makes the self-improvement loop converge — exactly the role a benchmark plays for kernels.

The autoresearch loop bolts onto the EXISTING graph and the EXISTING pg/MinIO store — no re-architecture. Each box below is incremental.

Already in place

- RLM deep-agent nodes (peek / delegate / build-up)
- web research (SearXNG + crawl4ai) → anchor clone
- error→fix lessons saved + recalled (kind='fix')
- pgvector recall + MinIO blobs, live ingest per step
- verilog_check deterministic gates + self-correct loop

Autoresearch upgrade

- research-on-failure: search the exact error string
- structured reflection: symptom→broken→fix template
- research budget controller (how deep to dig per step)
- lesson dedup + ranking by past usefulness
- memory-keeper: distil lessons → topic knowledge.md

Stretch

- clone & study auto-kernel / self-improving-agent repos
- into the store as 'method' knowledge
- auto-curated IP library feedback (this hardening run)
- cross-design transfer: reuse a verified sub-block
- offline self-play: regenerate past designs, keep wins

One-line summary

GarudaChip is already a recursive, memory-backed RTL agent. 'Autoresearch' makes its research PROACTIVE and its failures REFLECTIVE, and routes both through the pg/MinIO store— so every design it builds makes the next one smarter, the same way auto-kernel generators improve with use.